

Number of Islands

Given an $n \times m$ grid `grid[][]` consisting of 'L' (land) and 'W' (water), we need to count the total number of **islands** present in the grid without modifying the original grid.

An island is defined as a group of connected 'L' cells that are adjacent **horizontally, vertically, or diagonally**, and surrounded by water or the boundary of the grid.

Examples:

Input: `grid[][] = [['L', 'L', 'W', 'W', 'W'],`
 `['W', 'L', 'W', 'W', 'L'],`
 `['L', 'W', 'W', 'L', 'L'],`
 `['W', 'W', 'W', 'W', 'W'],`
 `['L', 'W', 'L', 'L', 'W']]`

Output: 4

Explanation: The image below shows all the 4 islands.

L	L	W	W	W
W	L	W	W	L
L	W	W	L	L
W	W	W	W	W
L	W	L	L	W

Input: `grid[][] = [['W', 'L', 'L', 'L', 'W', 'W', 'W'],
['W', 'W', 'L', 'L', 'W', 'L', 'W']]`

Output: 2

Explanation: The image below shows all the 2 islands in the graph

W	L	L	L	W	W	W
W	W	L	L	W	L	W

[Approach 1] Using DFS and Additional Matrix - $O(n*m)$ Time and $O(n*m)$ Space

The main idea is to explore each land cell ('L') using DFS and mark all connected land cells that belong to the same island. To avoid modifying the original grid, we maintain a separate visited matrix that keeps track of which cells have already been explored. The DFS explores all 8 possible directions (up, down, left, right, and 4 diagonals), marking every connected 'L' cell as visited. This ensures that all parts of the current island are counted once. Each time we encounter an unvisited land cell, it represents the start of a new island, and we perform DFS to mark all cells of that island. By the end, the total count represents the number of distinct islands in the grid.

// C++ Implementation.

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
// Checks if the given cell (r, c) can be visited
```

```
bool isSafe(vector<vector<char>> &grid, int r, int c, vector<vector<bool>> &visited)
```

```
{
```

```
    int n = grid.size();
```

```
    int m = grid[0].size();
```

```
    // Cell is within bounds, contains land ('L'), and is not yet visited
```

```
    return (r >= 0 && r < n && c >= 0 && c < m && grid[r][c] == 'L' && !visited[r][c]);
```

```
}
```

```
// Performs DFS to mark all connected land cells
```

```
void dfs(vector<vector<char>> &grid, int r, int c, vector<vector<bool>> &visited)
```

```
{
```

```
    // Mark current cell as visited
```

```
    visited[r][c] = true;
```

```
    // All 8 possible directions (vertical, horizontal, diagonal)
```

```
    vector<int> dr = {-1, -1, -1, 0, 0, 1, 1, 1};
```

```
    vector<int> dc = {-1, 0, 1, -1, 1, -1, 0, 1};
```

```

// Explore all connected neighbours
for (int k = 0; k < 8; k++)
{
    int nr = r + dr[k];
    int nc = c + dc[k];

    if (isSafe(grid, nr, nc, visited))
        dfs(grid, nr, nc, visited);
}
}

// finding number of distinct islands in the grid
int countIslands(vector<vector<char>> &grid)
{
    int n = grid.size();
    int m = grid[0].size();

    // Matrix to track visited cells
    vector<vector<bool>> visited(n, vector<bool>(m, false));

    int islands = 0;

    // Traverse every cell in the grid

```

```

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < m; j++)
    {
        // Start a new DFS when an unvisited land cell is found
        if (grid[i][j] == 'L' && !visited[i][j])
        {
            dfs(grid, i, j, visited);
            islands++;
        }
    }
}

return islands;
}

```

```

int main()
{
    vector<vector<char>> grid = {
        {'L', 'W', 'W', 'W', 'W'},
        {'W', 'L', 'W', 'W', 'L'},
        {'L', 'W', 'W', 'L', 'L'},
        {'W', 'W', 'W', 'W', 'W'},
    }
}

```

```

        {'L', 'W', 'L', 'L', 'W'});

// printing the number of islands
cout << countIslands(grid) << endl;

return 0;
}

```

[Approach 2] Using Breadth First Search - $O(n*m)$ time and $O(n*m)$ space

The idea is to use Breadth First Search (BFS) to explore all connected land cells for each island. We traverse the entire grid, and whenever we encounter an unvisited land cell ('L'), we perform a BFS starting from that cell to explore its entire connected component. During BFS, we visit all cells connected horizontally, vertically, or diagonally to the current land cell. To ensure we don't revisit the same cell multiple times, we maintain a separate boolean matrix to mark visited cells instead of modifying the original grid. Each BFS traversal explores one complete island, after which we increment the island counter. Repeating this for all cells ensures that every island is counted exactly once, and the final counter gives the total number of islands in the grid.

// C++ implementation.

```

#include <iostream>

#include <vector>

#include <queue>

using namespace std;

```

```

// Check if the cell (r, c) is valid for BFS traversal
// It must lie within grid bounds, contain land ('L'), and not be visited yet
bool isSafe(vector<vector<char>>& grid, int r, int c, vector<vector<bool>>&
visited) {
    int n = grid.size();
    int m = grid[0].size();
    return (r >= 0 && r < n && c >= 0 && c < m && grid[r][c] == 'L' && !visited[r][c]);
}

```

```

// Perform BFS to traverse all connected land cells (forming one island)
void bfs(vector<vector<char>>& grid, vector<vector<bool>>& visited, int
startR, int startC) {
    // Possible 8 directions (vertical, horizontal, and diagonal)
    vector<int> dRow = {-1, -1, -1, 0, 0, 1, 1, 1};
    vector<int> dCol = {-1, 0, 1, -1, 1, -1, 0, 1};

    queue<pair<int, int>> q;
    q.push({startR, startC});
    visited[startR][startC] = true;

```

```

// Explore all reachable land cells for this island
while (!q.empty()) {
    auto [r, c] = q.front();

```

```

q.pop();

// Check all 8 neighbors of the current cell
for (int k = 0; k < 8; k++) {
    int newR = r + dRow[k];
    int newC = c + dCol[k];
    if (isSafe(grid, newR, newC, visited)) {
        visited[newR][newC] = true;
        q.push({newR, newC});
    }
}
}

}

// Count the total number of islands in the grid
int countIslands(vector<vector<char>>& grid) {
    int n = grid.size();
    int m = grid[0].size();
    vector<vector<bool>> visited(n, vector<bool>(m, false));
    int islandCount = 0;

    // Traverse every cell in the grid
    for (int r = 0; r < n; r++) {

```



```

for (int c = 0; c < m; c++) {

    // If an unvisited land cell is found, start BFS for that island
    if (grid[r][c] == 'L' && !visited[r][c]) {
        bfs(grid, visited, r, c);
        islandCount++;
    }
}

return islandCount;
}

int main() {
    vector<vector<char>> grid = {
        {'L', 'L', 'W', 'W', 'W'},
        {'W', 'L', 'W', 'W', 'L'},
        {'L', 'W', 'W', 'L', 'L'},
        {'W', 'W', 'W', 'W', 'W'},
        {'L', 'W', 'L', 'L', 'W'}
    };

    cout << countIslands(grid) << endl;
}

```

```
    return 0;  
}
```

Output

4